

```

const questions = [
  { q: "This lemonade is very ____ and sour!", options: ["Zesty", "Synthetic", "Wholesome"],
    correct: 0, exp: "Zesty describes a sharp, pleasant, citrus-like flavor.", vocab: "Zesty" },
  { q: "That designer coffee is a bit too ____ for my budget.", options: ["Refreshing", "Pricey",
    "Energetic"], correct: 1, exp: "Pricey is an informal way to say expensive.", vocab: "Pricey" },
  { q: "A cold drink on a hot day is very ____.", options: ["Refreshing", "Cloying", "Superficial"],
    correct: 0, exp: "Refreshing means providing fresh energy or spirit.", vocab: "Refreshing" },
  { q: "This juice has ____ sweeteners, not real fruit.", options: ["Synthetic", "Rejuvenated",
    "Wholesome"], correct: 0, exp: "Synthetic means made by chemical synthesis, not natural.",
    vocab: "Synthetic" },
  { q: "The morning shift barista is always so ____.", options: ["Exorbitant", "Energetic",
    "Cloying"], correct: 1, exp: "Energetic means showing or involving great activity.", vocab:
    "Energetic" },
  { q: "The syrup was so sweet it became ____.", options: ["Wholesome", "Cloying", "Zesty"],
    correct: 1, exp: "Cloying describes something excessively sweet to the point of being
    unpleasant.", vocab: "Cloying" },
  { q: "His knowledge of coffee beans is quite ____.", options: ["Superficial", "Rejuvenated",
    "Exorbitant"], correct: 0, exp: "Superficial means existing or occurring at or on the surface; not
    deep.", vocab: "Superficial" },
  { q: "After a nap, the barista felt ____.", options: ["Synthetic", "Rejuvenated", "Pricey"],
    correct: 1, exp: "Rejuvenated means to feel young or energetic again.", vocab: "Rejuvenated" },
  { q: "The price for that rare bean was ____!", options: ["Exorbitant", "Wholesome", "Zesty"],
    correct: 0, exp: "Exorbitant describes a price that is unreasonably high.", vocab: "Exorbitant" },
  { q: "We only serve ____, organic ingredients here.", options: ["Cloying", "Wholesome",
    "Superficial"], correct: 1, exp: "Wholesome means conducive to or suggestive of good health.",
    vocab: "Wholesome" }
];

```

```

const game = {
  currentQ: 0,
  p1Score: 0,
  p2Score: 0,
  p1Selection: 0,
  p2Selection: 0,
  timer: 10,
  timerInterval: null,
  isLocked: false,

  init() {
    window.addEventListener('keydown', (e) => this.handleInput(e));
    this.loadQuestion();
  },

  loadQuestion() {

```

```

    if (this.currentQ >= questions.length) return this.endGame();

    this.isLocked = false;
    const qData = questions[this.currentQ];
    document.getElementById('question-text').innerText = qData.q;
    const container = document.getElementById('options-container');
    container.innerHTML = "";

    qData.options.forEach((opt, i) => {
        const div = document.createElement('div');
        div.className = 'option';
        div.id = `opt-${i}`;
        div.innerText = opt;
        container.appendChild(div);
    });

    this.p1Selection = 0;
    this.p2Selection = 0;
    this.updateSelectionVisuals();
    this.startTimer();
},

startTimer() {
    this.timer = 10;
    clearInterval(this.timerInterval);
    this.timerInterval = setInterval(() => {
        this.timer -= 0.1;
        document.getElementById('timer-fill').style.width = (this.timer * 10) + '%';
        if (this.timer <= 0) this.resolveRound();
    }, 100);
},

handleInput(e) {
    if (this.isLocked) return;

    // Player 1 (W/S/D)
    if (e.key.toLowerCase() === 'w') this.p1Selection = Math.max(0, this.p1Selection - 1);
    if (e.key.toLowerCase() === 's') this.p1Selection = Math.min(2, this.p1Selection + 1);
    if (e.key.toLowerCase() === 'd') this.resolveRound('p1');

    // Player 2 (Arrows/Enter)
    if (e.key === 'ArrowUp') this.p2Selection = Math.max(0, this.p2Selection - 1);
    if (e.key === 'ArrowDown') this.p2Selection = Math.min(2, this.p2Selection + 1);
    if (e.key === 'ArrowLeft' || e.key === 'Enter') this.resolveRound('p2');
}

```

```

    this.updateSelectionVisuals();
},

updateSelectionVisuals() {
    document.querySelectorAll('.option').forEach((el, i) => {
        el.classList.remove('p1-selected', 'p2-selected', 'both-selected');
        if (i === this.p1Selection && i === this.p2Selection) el.classList.add('both-selected');
        else if (i === this.p1Selection) el.classList.add('p1-selected');
        else if (i === this.p2Selection) el.classList.add('p2-selected');
    });
},

resolveRound(firstMover) {
    this.isLocked = true;
    clearInterval(this.timerInterval);

    const qData = questions[this.currentQ];
    const p1Correct = this.p1Selection === qData.correct;
    const p2Correct = this.p2Selection === qData.correct;

    // Scoring: Base 100 + Speed Bonus (up to 50)
    const bonus = Math.floor(this.timer * 5);
    if (p1Correct) this.p1Score += (100 + bonus);
    if (p2Correct) this.p2Score += (100 + bonus);

    document.getElementById('p1-score').innerText = this.p1Score;
    document.getElementById('p2-score').innerText = this.p2Score;

    this.showExplanation(qData.exp);
},

showExplanation(text) {
    const screen = document.getElementById('explanation-screen');
    document.getElementById('explanation-text').innerText = text;
    screen.classList.remove('hidden');
},

nextQuestion() {
    document.getElementById('explanation-screen').classList.add('hidden');
    this.currentQ++;
    this.loadQuestion();
},

```

```
endGame() {  
  const screen = document.getElementById('result-screen');  
  const winner = this.p1Score > this.p2Score ? "Smart Barista Wins!" :  
    this.p2Score > this.p1Score ? "Villain Barista Wins!" : "It's a Tie!";  
  document.getElementById('winner-text').innerText = winner;  
  screen.classList.remove('hidden');  
}  
};  
  
game.init();
```